
INFRASTRUCTURE ET ARCHITECTURE DE DONNÉES

Séance 1 — Conteneurs et flux : Docker, Compose et premier producteur Kafka

ÉNONCÉ APPRENANT

Sommaire

Séance 1 — Conteneurs et flux : Docker, Compose et premier producteur Kafka	3
En-tête de séance	3
Contexte métier : HealthStream ICU Monitor	4
Objectifs pédagogiques	4
Arborescence de travail	4
Partie 1 — Docker : image, conteneur, couches	6
1.1 Image et conteneur : la distinction	6
1.2 Écrire un Dockerfile exploitant le cache	6
1.3 Construire, lancer, inspecter	7
Partie 2 — Docker Compose : du <code>docker run</code> au service déclaré	9
2.1 Pourquoi Compose	9
2.2 Premier <code>docker-compose.yml</code>	9
2.3 Réseau : résolution par nom de service	10
2.4 <code>healthcheck</code> et dépendances conditionnelles	10
Partie 3 — Kafka en local, en mode KRaft	12
3.1 Les trois notions à retenir	12
3.2 Ajouter le broker au Compose	12
3.3 Créer le topic et inspecter ses partitions	14
3.4 Produire et lire en ligne de commande	14
Partie 4 — Producteur Python de signes vitaux	16
4.1 Format des messages	16
4.2 Pourquoi la clé de partitionnement compte	16
4.3 Écrire le producteur	16
4.4 Exécuter depuis l'hôte	18
4.5 Exécuter via Compose, avec dépendance conditionnelle	18
Partie 5 — Vérification et mesure	20
5.1 Débit de production	20
5.2 Coupure du broker pendant la production	20
Nettoyage	22
Livrable	22
Ce que vous emportez vers la séance 2	22

Séance 1 — Conteneurs et flux : Docker, Compose et premier producteur Kafka

Module : Infrastructure et architecture de données **Niveau** : Master 2 — YNOV Campus Montpellier **Année** : 2026-2027 **Document** : énoncé apprenant

En-tête de séance

Élément	Détail
Durée	3 h 30
Modalité	TP individuel sur poste, rendu en fin de séance
Prérequis matériel	Docker Desktop 24+ ou Colima/Rancher Desktop, 8 Go de RAM libre, 10 Go d'espace disque
Prérequis logiciel	<code>docker</code> et <code>docker compose</code> v2 fonctionnels, Python 3.11 ou 3.12 sur le poste, un éditeur de texte
Prérequis de connaissances	Ligne de commande Unix, bases Python (fonctions, dictionnaires, boucles), notion de JSON
Livrable attendu	Une archive <code>S1_<NOM>_<Prenom>.zip</code> contenant le dossier <code>s1-vitals-stream/</code> complet (voir section « Livrable »)

Convention de lecture

Cet énoncé ne vous donne pas le contenu intégral des fichiers à écrire. Pour chaque artefact, vous trouverez trois choses : un paragraphe qui décrit **la logique** attendue, un tableau qui liste **les éléments** à produire avec leur rôle et leur contrainte, et un **extrait court** portant sur le seul point délicat. Les commandes à taper, elles, sont données telles quelles. Les valeurs à décider apparaissent en `_____` ou dans un tableau de décisions ; elles se justifient dans `MESURES.md`.

Vérification des prérequis (à faire avant de commencer)

```
docker --version           # moteur Docker, doit être >= 24
docker compose version     # plugin Compose v2 (commande sans tiret), >= v2.20
python3 --version          # Python du poste hôte
docker info --format '{{.ServerVersion}} / {{.OperatingSystem}}' # le démon répond-il ?
```

Notez ici les trois versions obtenues, elles vous serviront à expliquer d'éventuels écarts de comportement avec vos camarades :

Outil	Version relevée sur votre poste
Docker Engine	<code>_____</code>
Docker Compose	<code>_____</code>
Python	<code>_____</code>

Contexte métier : HealthStream ICU Monitor

Un service de réanimation (ICU, *Intensive Care Unit*) de 12 lits souhaite se doter d'un système de supervision temps réel. Chaque lit est équipé d'un moniteur qui mesure en continu la fréquence cardiaque, la saturation en oxygène et la pression artérielle du patient. Aujourd'hui ces valeurs restent affichées localement au pied du lit : personne ne peut les corréliser entre patients, ni détecter une dégradation lente, ni déclencher une alerte à distance.

Le projet **HealthStream ICU Monitor**, qui sera assemblé lors des séances 4 à 6, consiste à construire cette chaîne complète : un simulateur rejoue les signes vitaux issus du jeu de données public **MIMIC-IV**, les publie dans un bus de messages, un moteur de traitement de flux calcule des fenêtres glissantes et détecte les seuils critiques, le tout déployé sur Kubernetes avec observabilité et enrichissement des alertes par un LLM.

Vous ne construirez pas cette chaîne aujourd'hui. Vous construisez sa **toute première brique** : l'environnement conteneurisé et le producteur qui alimente le bus. Ce que vous produisez en séance 1 n'est pas un exercice jetable — le `docker-compose.yml` et le producteur Python que vous écrivez aujourd'hui seront **repris tels quels en séance 2** (pour y brancher un consommateur), **étendus en séance 3** (orchestration Airflow), et **deviendront le simulateur de signes vitaux MIMIC-IV du projet final**. Le nom du topic, le format JSON des messages et la clé de partitionnement que vous mettez en place ici sont ceux du projet. Soignez-les.

Objectifs pédagogiques

À l'issue de cette séance, vous savez :

1. Distinguer une image d'un conteneur, et expliquer ce qu'est une couche d'image.
2. Écrire un `Dockerfile` qui exploite le cache de build et n'exécute pas le processus en root.
3. Traduire une commande `docker run` en service `docker-compose.yml`, avec réseau, volume nommé et `healthcheck`.
4. Expliquer la différence entre `depends_on` seul et `depends_on` avec `condition: service_healthy`.
5. Lancer un broker Kafka en mode KRaft, créer un topic, et manipuler les notions de topic, partition et offset.
6. Écrire un producteur Python qui publie du JSON clé/valeur, et justifier le choix de la clé de partitionnement.
7. Mesurer un débit de production et observer le comportement du producteur quand le broker disparaît.

Arborescence de travail

Créez le dossier de travail suivant. Vous le remplirez au fil des parties.

```
mkdir -p ~/s1-vitals-stream/producer # arborescence du projet de la séance
cd ~/s1-vitals-stream
pwd # toutes les commandes du TP partent d'ici
```

Arborescence cible en fin de séance :

```
s1-vitals-stream/  
├─ docker-compose.yml      # artefact principal, réutilisé dès la séance 2  
├─ MESURES.md              # vos relevés chiffrés (parties 1, 3, 5)  
└─ producer/  
    ├─ Dockerfile          # image du producteur  
    ├─ .dockerignore  
    ├─ requirements.txt  
    └─ producer.py         # producteur de signes vitaux
```

Partie 1 — Docker : image, conteneur, couches

1.1 Image et conteneur : la distinction

Une **image** est un modèle en lecture seule : un empilement de couches de système de fichiers, plus des métadonnées (commande de démarrage, variables d'environnement, utilisateur). Une image ne s'exécute pas, elle se stocke.

Un **conteneur** est une instance d'exécution d'une image : l'empilement de couches de l'image, plus une couche supérieure inscriptible propre au conteneur, plus un processus isolé (namespaces) et limité (cgroups).

Conséquence directe : d'une même image, on peut lancer dix conteneurs ; chacun a sa propre couche inscriptible, donc les fichiers écrits par l'un sont invisibles pour les autres, et disparaissent avec lui.

Vérifiez-le :

```
docker pull python:3.12-slim # récupère l'image de base, ne lance rien
docker images python
# Un premier conteneur écrit un fichier dans SA couche inscriptible
docker run --name c1 python:3.12-slim sh -c "echo patient-001 > /tmp/marqueur; cat /tmp/marqueur"
# Un second conteneur, depuis LA MÊME image, cherche ce fichier
docker run --name c2 python:3.12-slim sh -c "cat /tmp/marqueur || echo 'fichier absent'"
```

Question 1.1 — Le second conteneur voit-il le fichier écrit par le premier ? Notez la réponse observée et la raison en une phrase dans `MESURES.md`.

```
docker rm c1 c2 # nettoyer les deux conteneurs de démonstration
```

1.2 Écrire un Dockerfile exploitant le cache

Docker construit une image couche par couche : **une instruction du Dockerfile produit une couche**. Au rebuild, Docker réutilise une couche en cache tant que l'instruction et son contexte n'ont pas changé ; dès qu'une couche est invalidée, **toutes les suivantes le sont aussi**.

Règle qui en découle : placez ce qui change rarement (installation des dépendances) **avant** ce qui change à chaque édition (votre code source). Copier `requirements.txt` seul, installer, puis copier le code, permet de ne pas réinstaller les dépendances à chaque modification d'une ligne de Python.

Deuxième règle : un conteneur ne doit pas exécuter son processus en `root`. Si le processus est compromis, un `root` dans le conteneur partage l'UID 0 avec l'hôte, ce qui élargit inutilement la surface d'attaque en cas d'évasion ou de montage mal configuré.

Deux fichiers courts à créer d'abord, sans difficulté :

- `producer/requirements.txt` : une seule dépendance, `confluent-kafka==2.6.1`.
- `producer/.dockerignore` : `__pycache__`, `*.pyc`, `.venv/`, `.git/`, `MESURES.md`. Il réduit le contexte de build envoyé au démon et évite d'invalider le cache pour rien.

Logique du `Dockerfile`. Le fichier se lit comme une suite d'étapes ordonnées par leur fréquence de changement : on part d'une image de base figée, on fixe l'environnement d'exécution de Python, on installe les dépendances à partir du seul fichier qui les décrit, **puis** on copie le code, et on ne bascule sur un compte non privilégié qu'à la fin, une fois les droits posés. L'ordre n'est pas cosmétique : il détermine ce que le cache peut réutiliser et si le processus final tourne en root.

Éléments attendus dans `producer/Dockerfile`, dans cet ordre :

Ordre	Instruction	Rôle	Contrainte
1	<code>FROM</code>	image de base	variante légère, version figée — jamais <code>latest</code>
2	<code>ENV</code>	comportement de Python	<code>PYTHONDONTWRITEBYTECODE=1</code> et <code>PYTHONUNBUFFERED=1</code> : sans le second, <code>docker logs</code> reste muet
3	<code>WORKDIR</code>	répertoire applicatif	un chemin absolu court, à décider ; toutes les instructions suivantes s'y exécutent
4	<code>COPY</code>	le fichier de dépendances, seul	étape qui change rarement
5	<code>RUN pip install</code>	installation	<code>--no-cache-dir</code> , pour ne pas embarquer le cache pip dans la couche
6	<code>COPY</code>	le code applicatif	étape qui change souvent, donc après l'installation
7	<code>RUN use-radd</code>	compte non privilégié	UID <code>10001</code> , puis <code>chown -R</code> du répertoire de travail
8	<code>USER</code>	bascule définitive	placée avant <code>CMD</code> , sinon le processus tourne en root
9	<code>CMD</code>	commande par défaut	<code>["python", "producer.py"]</code>

Extrait sur le point délicat — la coupure du cache au bon endroit :

```
# Deux COPY séparés par l'installation : c'est là que se joue le cache.
COPY _____ ./
RUN pip install --no-cache-dir -r requirements.txt
COPY _____ ./
# Un unique "COPY . ." placé avant le RUN invaliderait l'installation
# des dépendances à chaque modification d'une ligne de Python.
```

Note : `producer.py` n'existe pas encore. Créez un fichier provisoire pour pouvoir construire dès maintenant :

```
echo 'print("producteur: squelette OK")' > producer/producer.py
```

1.3 Construire, lancer, inspecter

```
IMAGE=_____ # nom:tag explicite, évitez "latest"
docker build -t "$IMAGE" ./producer # -t = nom:tag ; le point final = contexte de build
```

```
docker run --rm "$IMAGE"           # conteneur éphémère
docker run --rm "$IMAGE" id        # doit afficher uid=10001, PAS uid=0
```

Inspection des couches :

```
docker history "$IMAGE"           # taille et instruction d'origine de chaque couche
docker images --format "{{.Repository}}:{{.Tag}} {{.Size}}" | grep "$IMAGE%:.*"
docker image inspect "$IMAGE" --format 'User={{.Config.User}} Cmd={{.Config.Cmd}}'
```

Mesure 1 — Complétez dans `MESURES.md` :

Grandeur	Valeur relevée
Taille de <code>python:3.12-slim</code>	
Taille de votre image finale	
Nombre de couches non vides dans <code>docker history</code>	
Couche la plus volumineuse (instruction + taille)	

Expérience du cache — modifiez une ligne de `producer/producer.py` (changez le texte affiché), puis relancez le build en chronométrant :

```
time docker build -t "$IMAGE" ./producer # rebuild après modification du code seul
```

Puis modifiez `producer/requirements.txt` (ajoutez une ligne de commentaire `# test cache`) et rebuildz de nouveau en chronométrant.

Mesure 2 — Complétez (durée relevée, puis nombre d'étapes affichées en `CACHED`) :

Scénario de rebuild	Durée / étapes <code>CACHED</code>
Aucune modification	
Modification de <code>producer.py</code>	
Modification de <code>requirements.txt</code>	

Question 1.2 — Si vous aviez écrit `COPY . ./` avant le `RUN pip install`, lequel des trois scénarios ci-dessus changerait, et dans quel sens ? Répondez en deux phrases dans `MESURES.md`. (N'oubliez pas de retirer la ligne `# test cache`.)

Partie 2 — Docker Compose : du `docker run` au service déclaré

2.1 Pourquoi Compose

Une commande `docker run` complète devient vite illisible et non reproductible :

```
# Ce qu'on veut éviter : tout est dans la ligne de commande, rien n'est versionné
docker run -d --name producer --network vitals-net -e KAFKA_BOOTSTRAP=kafka:9092 \
  -e VITALS_TOPIC=mimic-vitals -v vitals-logs:/app/logs --restart unless-stopped mon-image:0.1
```

Compose déplace cette configuration dans un fichier YAML versionné : même environnement pour tout le monde, `docker compose up` pour tout lancer, `docker compose down` pour tout retirer.

2.2 Premier `docker-compose.yml`

Créez `docker-compose.yml` à la racine de `s1-vitals-stream/`. Pour l'instant, un seul service : le producteur (qui ne fait encore qu'afficher un message).

Logique du fichier. Un fichier Compose se lit en trois temps. D'abord la clé `services:`, où chaque service décrit *un* conteneur : d'où vient son image, ce qu'il reçoit par variables d'environnement, ce qu'il monte, à quel réseau il est attaché. Ensuite les sections racine `volumes:` et `networks:`, qui **déclarent** les ressources nommées : un volume ou un réseau cité dans un service mais absent de ces sections fait échouer le fichier. Enfin, rien n'est implicite : ce qui n'est pas écrit n'existe pas, et c'est précisément ce qui rend le fichier reproductible d'un poste à l'autre.

Éléments attendus pour ce premier fichier :

Élément	Rôle	Contrainte
service <code>producer</code>	le conteneur du producteur	construit depuis le Dockerfile de la partie 1, via <code>build.context</code>
<code>container_name</code>	nom fixe du conteneur	<code>vitals-producer</code> , pratique pour <code>docker logs</code> et <code>docker exec</code>
<code>environment.KAFKA_BOOTSTRAP</code>	adresse du broker lue par le code Python	<code>kafka:9092</code> — nom de service, pas <code>localhost</code>
<code>environment.VITALS_TOPIC</code>	topic cible	le topic du projet, imposé (voir 4.1)
<code>volumes</code> du service	journaux applicatifs	un volume nommé monté sur <code>/app/logs</code> , pas un bind mount
<code>networks</code> du service	rattachement réseau	le réseau applicatif dédié du projet
section racine <code>volumes:</code>	déclaration du volume nommé	obligatoire, sinon Compose refuse le fichier
section racine <code>networks:</code>	déclaration du réseau	<code>driver: bridge</code>

Décisions à prendre et à reporter dans `MESURES.md` :

Décision	Valeur retenue
Nom du volume nommé des journaux	
Nom du réseau applicatif	(il servira à tous les services jusqu'à la séance 6)
Valeur de <code>VITALS_TOPIC</code>	(voir le tableau 4.1)

Extrait sur le point délicat — un nom de ressource apparaît **deux fois**, dans le service et dans la déclaration racine :

```
volumes:
  - :/app/logs      # volume NOMMÉ, et non ./logs:/app/logs
networks:
  - 
# ... puis, au niveau racine du fichier, en fin de document :
volumes:
  :                # déclaration du volume nommé utilisé ci-dessus
networks:
  :
  driver: bridge
```

```
docker compose up -d --build    # construire et lancer l'ensemble décrit par le fichier
docker compose ps               # état des services
docker compose logs producer   # journaux du service producer
```

Points à vérifier et à noter :

```
docker network ls | grep vitals  # nom RÉEL du réseau (préfixé par le nom du projet)
docker volume ls | grep vitals   # nom RÉEL du volume
# Où Docker stocke-t-il ce volume sur l'hôte ? Utilisez le nom COMPLET relevé ci-dessus.
docker volume inspect <nom_complet_du_volume> --format '{{.Mountpoint}}'
```

Question 2.1 — Pourquoi utilise-t-on ici un volume **nommé** plutôt qu'un bind mount `./logs:/app/logs` ?
Donnez deux arguments dans `MESURES.md`.

2.3 Réseau : résolution par nom de service

Sur un réseau Compose défini par l'utilisateur, le DNS interne résout le **nom du service**. C'est la raison pour laquelle le producteur contactera le broker via `kafka:9092` et non via une adresse IP.

```
# Conteneur jetable attaché au même réseau, pour tester la résolution
# (remplacez par le nom RÉEL du réseau relevé plus haut, préfixe de projet inclus)
docker run --rm --network <nom_complet_du_reseau> busybox nslookup producer
```

2.4 `healthcheck` et dépendances conditionnelles

Un `healthcheck` est une commande que Docker exécute périodiquement **à l'intérieur** du conteneur. Son code de sortie détermine l'état : `0` → `healthy`, autre → `unhealthy` (après `retries` échecs consécutifs). Sans `healthcheck`, un conteneur est seulement `running` : Docker sait que le processus vit, pas qu'il est prêt à servir.

C'est exactement la différence qui compte pour les dépendances :

Écriture	Ce que Compose attend avant de démarrer le service dépendant
<code>depends_on: [kafka]</code>	que le conteneur <code>kafka</code> soit démarré (processus lancé) — rien de plus
<code>depends_on: kafka: condition: service_healthy</code>	que le <code>healthcheck</code> de <code>kafka</code> soit passé au vert

Un broker Kafka met plusieurs secondes à charger ses métadonnées et à ouvrir son port. Avec `depends_on` seul, le producteur démarre pendant ce délai et échoue à se connecter. C'est le piège classique de la partie 4 — vous le provoquerez volontairement.

Ajoutez un `healthcheck` au service `producer` pour manipuler la syntaxe. Cinq clés, dont deux que vous décidez :

Clé	Rôle	Valeur
<code>test</code>	commande exécutée dans le conteneur ; code de sortie 0 = <code>healthy</code>	ici un test trivial : <code>python -c "import sys; sys.exit(0)"</code>
<code>interval</code>	délai entre deux exécutions du test	à décider, à justifier
<code>timeout</code>	au-delà, une exécution du test est en échec	<code>5s</code>
<code>retries</code>	échecs consécutifs avant le passage <code>unhealthy</code>	à décider, à justifier
<code>start_period</code>	période de grâce au démarrage : les échecs n'y sont pas comptés	<code>10s</code>

```
healthcheck:
  test: ["CMD", "python", "-c", "import sys; sys.exit(0)"]
  interval: _____ # trop court : le test charge le conteneur pour rien
  timeout: 5s
  retries: _____ # trop grand : un service mort reste "healthy" longtemps
  start_period: 10s
```

```
docker compose up -d # recréer le service avec son healthcheck
docker compose ps    # la colonne STATUS passe de "health: starting" à "healthy"
```

Mesure 3 — Combien de secondes s'écoulent entre le démarrage du conteneur et le passage à `healthy` ? Relevez-le :

```
# Historique des exécutions du healthcheck, avec horodatage et code de sortie
docker inspect vitals-producer --format '{{json .State.Health}}'
```

Notez : délai observé avant `healthy` = _____ s.

Partie 3 — Kafka en local, en mode KRaft

3.1 Les trois notions à retenir

- **Topic** : un flux nommé et persistant. Ici `mimic-vitals` transportera les relevés de signes vitaux.
- **Partition** : un topic est découpé en N partitions. Chaque partition est un **journal ordonné en append-only**. L'ordre des messages n'est garanti **qu'à l'intérieur d'une partition**, jamais à l'échelle du topic.
- **Offset** : la position d'un message dans sa partition, un entier croissant. Un message est identifié par le triplet (topic, partition, offset). Lire, c'est avancer un curseur d'offset — les messages ne sont pas consommés au sens « retirés ».

KRaft (*Kafka Raft*) remplace ZooKeeper depuis Kafka 3.3 : le broker héberge lui-même le quorum de méta-données. Un seul conteneur suffit, ce qui est notre cas ici (un nœud jouant les deux rôles `broker` et `controller`).

3.2 Ajouter le broker au Compose

Ajoutez un service `kafka` **au-dessus** du service `producer` dans `docker-compose.yml`.

Logique du service. Un broker Kafka en KRaft se configure presque entièrement par variables d'environnement, et celles-ci se regroupent en cinq familles qu'il faut traiter dans l'ordre : *qui suis-je* (identité du nœud et quorum de contrôleurs), *sur quoi j'écoute* (les listeners), *quelle adresse j'annonce aux clients* (les listeners annoncés), *combien de copies je garde* (réplication — tout à 1 puisqu'il n'y a qu'un nœud), et *où j'écris mes journaux* (répertoire de données, à poser sur un volume nommé pour survivre à un `down`). S'y ajoutent un `healthcheck` qui interroge l'API du broker, la publication d'un port vers l'hôte, et le rattachement au réseau applicatif.

Éléments attendus, par famille :

Fa-mille	Clés	Valeur ou contrainte
Image et nom	<code>image</code> , <code>container_name</code>	<code>confluentinc/cp-kafka:7.8.0</code> (version figée), <code>vitals-kafka</code>
Identité KRaft	<code>KAFKA_NODE_ID</code> , <code>KAFKA_PROCESS_ROLES</code> , <code>KAFKA_CONTROLLER_QUORUM_VOTERS</code> , <code>CLUSTER_ID</code>	nœud <code>1</code> ; ce nœud porte les deux rôles ; quorum <code>1@kafka:9093</code> ; <code>CLUSTER_ID: MKU30EVBNTcwNT-JENDM2Qg</code> (fourni, ne pas modifier)
Listeners	<code>KAFKA_LISTENERS</code> , <code>KAFKA_ADVERTISED_LISTENERS</code> , <code>KAFKA_LISTENER_SECURITY_PROTOCOL_MAP</code> , <code>KAFKA_CONTROLLER_LISTENER_NAMES</code> , <code>KAFKA_INTER_BROKER_LISTENER_NAME</code>	trois listeners : <code>PLAINTEXT</code> sur 9092 (clients internes), <code>CONTROLLER</code> sur 9093, <code>PLAINTEXT_HOST</code> sur 29092 (clients du poste) ; les trois en <code>PLAINTEXT</code> dans la map ; contrôleur = <code>CONTROLLER</code> , inter-broker = <code>PLAINTEXT</code>
Réplication	<code>KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR</code> , <code>KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR</code> , <code>KAFKA_TRANSACTION_STATE_LOG_MIN_ISR</code>	un seul broker, donc la même valeur minimale partout

Fa-mille	Clés	Valeur ou contrainte
Dé-mar-rage et garde-fous	KAF- KA_GROUP_INITIAL_REBALANCE_DELAY_MS , KAFKA_AUTO_CREATE_TOPICS_ENABLE	0 (pas de rééquilibrage différé en développement) ; création implicite de topics désactivée , pour que toute faute de frappe sur un nom se voie
Don-nées	KAFKA_LOG_DIRS + volumes	/var/lib/kafka/data , monté sur un volume nommé déclaré en section racine
Santé	healthcheck	kafka-broker-api-versions --bootstrap-server localhost:9092 ; interval: 10s , timeout: 10s , retries: 10 , start_period: 20s
Ré-seau et port	networks , ports	même réseau que le producteur ; publication "<port_hote>:29092"

Décisions à prendre :

Décision	Valeur retenue
Port hôte publié en face de 29092	
Valeur de KAFKA_PROCESS_ROLES	
Facteur de réplication des topics internes	
KAFKA_AUTO_CREATE_TOPICS_ENABLE	
Nom du volume nommé des journaux Kafka	

Extrait sur le point délicat — **listeners** et **listeners annoncés** ne sont pas la même chose :

```
# Ce sur quoi le broker écoute réellement (0.0.0.0 = toutes les interfaces)
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093,PLAINTEXT_HOST://
0.0.0.0:29092
# Ce que le broker RENVOIE aux clients, et qu'ils utiliseront pour le rappeler
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://_____:9092,PLAINTEXT_HOST://localhost:29092
# Annoncer localhost sur le listener interne ferait échouer le producteur
# conteneurisé : il chercherait le broker dans son propre conteneur.
```

Déclarez le nouveau volume dans la section **volumes:** du fichier, puis lancez :

```
docker compose config      # valide la syntaxe et affiche la config résolue, sans rien démar-
rer
docker compose up -d kafka # démarrer uniquement le broker
docker compose ps
docker compose logs -f kafka # suivre le démarrage jusqu'à "healthy" (Ctrl-C pour quitter)
```

Mesure 4 — Temps écoulé entre `up -d kafka` et le passage à `healthy` : s.

3.3 Créer le topic et inspecter ses partitions

Les outils en ligne de commande de Kafka sont dans l'image : on les exécute dans le conteneur avec `docker compose exec`.

```
TOPIC=_____ # nom du topic du projet HealthStream, imposé (voir tableau 4.1)
# --partitions : nombre de journaux parallèles ; --replication-factor : copies par partition
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic "$TOPIC" --partitions _____ --replication-factor _____
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --list
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --describe --topic "$TOPIC"
```

Question 3.1 — Pourquoi `--replication-factor` ne peut-il pas valoir 2 ici ? Que renverrait Kafka ? Répondez dans `MESURES.md`.

3.4 Produire et lire en ligne de commande

```
# Producteur console avec clé : le séparateur déclaré sépare clé et valeur sur chaque ligne
docker compose exec -it kafka kafka-console-producer --bootstrap-server localhost:9092 \
  --topic "$TOPIC" --property "parse.key=true" --property "key.separator=:"
```

Saisissez ces lignes, une par une, puis quittez avec `Ctrl-D` :

```
P001:{"vital_type":"HR","value":88}
P002:{"vital_type":"HR","value":102}
P001:{"vital_type":"SP02","value":96}
P003:{"vital_type":"HR","value":71}
P001:{"vital_type":"ABP_SYS","value":118}
```

Relisez tout depuis le début, en affichant les métadonnées :

```
# --from-beginning repart de l'offset 0 de chaque partition ;
# les propriétés d'affichage exposent partition, offset et clé
docker compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic "$TOPIC" --from-beginning --property print.partition=true \
  --property print.offset=true --property print.key=true --timeout-ms 8000
```

Mesure 5 — Reportez la répartition observée (partition(s) atteinte(s), puis offsets) :

Clé	Partition(s) et offsets observés
P001	
P002	
P003	

Puis relevez les bornes de chaque partition :

```
# Offset du prochain message à écrire, pour chaque partition (-1 = fin de partition)
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --bootstrap-server localhost:9092 --topic "$TOPIC" --time -1
```

Question 3.2 — Les trois messages de `P001` sont-ils tous dans la même partition ? Les offsets se suivent-ils ? Que pouvez-vous en déduire sur la garantie d'ordre ? Répondez en trois phrases dans `MESURES.md`.

Partie 4 — Producteur Python de signes vitaux

4.1 Format des messages

C'est le format du projet HealthStream. Ne le modifiez pas : la séance 2 écrira un consommateur qui l'attend tel quel.

Champ	Type	Description
<code>patient_id</code>	string	Identifiant du patient, ex. <code>P001</code>
<code>event_time</code>	entier (ms epoch)	Horodatage du relevé
<code>vital_type</code>	string	<code>HR</code> , <code>SP02</code> ou <code>ABP_SYS</code>
<code>value</code>	float	Valeur mesurée
<code>unit</code>	string	<code>bpm</code> , <code>%</code> ou <code>mmHg</code>

La clé Kafka du message est le `patient_id` , encodé en UTF-8. Le topic est `mimic-vitals` .

4.2 Pourquoi la clé de partitionnement compte

Le producteur applique par défaut un partitionnement par hachage de la clé : `partition = hash(clé) mod nombre_de_partitions` . Deux conséquences :

1. **Tous les messages d'un même patient vont dans la même partition.** Comme l'ordre n'est garanti qu'à l'intérieur d'une partition, la série temporelle de chaque patient reste chronologique côté lecture. Sans clé (`key=None`), le producteur répartit les messages sur toutes les partitions : la fréquence cardiaque d'un même patient pourrait être lue dans le désordre, et la détection de tendance en séance 4 deviendrait fausse.
2. **Le parallélisme est borné par le nombre de partitions**, et l'équilibrage dépend de la distribution des clés. Une clé trop grossière (par exemple `vital_type` , qui ne prend que 3 valeurs) concentrerait la charge sur 3 partitions au maximum.

4.3 Écrire le producteur

Remplacez le contenu de `producer/producer.py` .

Logique du script. Le producteur suit quatre temps. (1) *Configuration* : toutes les valeurs variables — adresse du broker, topic, nombre de patients, intervalle d'échantillonnage — sont lues dans l'environnement avec une valeur par défaut, de sorte que le même fichier tourne depuis le poste hôte et depuis un conteneur sans une ligne de code modifiée. (2) *Construction du producteur* : un dictionnaire de configuration qui fixe le niveau d'acquiescement, le regroupement en lots, les réémissions et le partitionneur, puis un objet `Producer` . (3) *Boucle de production* : à chaque tour, un relevé par patient — type de signe vital tiré parmi les trois, valeur tirée dans la plage physiologique correspondante, horodatage en millisecondes — est publié avec le `patient_id` en clé, puis un `poll(0)` traite les accusés de réception en attente et le script attend l'intervalle. (4) *Arrêt propre* : un gestionnaire de signal lève un drapeau qui fait sortir de la boucle ; un `flush` borné vide le buffer avant de quitter et le script imprime un bilan (messages envoyés, non livrés, durée, débit) — c'est ce bilan que la partie 5 exploite.

Éléments attendus dans `producer/producer.py` :

Élément	Rôle	Contrainte
Constantes d'environnement	<code>KAFKA_BOOTSTRAP</code> , <code>VITALS_TOPIC</code> , <code>NB_PATIENTS</code> , <code>INTERVAL_S</code>	valeurs par défaut utilisables depuis l'hôte (<code>localhost:<port publié></code>)
Table <code>PLAGES</code>	plages physiologiques et unités	<code>HR</code> 55–115 <code>bpm</code> , <code>SP02</code> 90–100 <code>%</code> , <code>ABP_SYS</code> 95–145 <code>mmHg</code>
<code>gerer_signal(signum, frame)</code>	armé sur <code>SIGINT</code> et <code>SIGTERM</code>	lève un drapeau d'arrêt, ne quitte pas brutalement
<code>creer_producteur() -> Producer</code>	construit le producteur et sa configuration de fiabilité	voir le tableau de décisions ci-dessous
<code>ac-cuse_reception(err, msg)</code>	callback de livraison, appelé par <code>poll()</code> / <code>flush()</code>	affiche les échecs sur <code>stderr</code> ; ne tracez pas chaque succès, c'est illisible
<code>generer_releve(patient_id) -> dict</code>	produit un relevé conforme au format 4.1	<code>event_time</code> en millisecondes epoch, <code>value</code> arrondie à une décimale
<code>main() -> int</code>	boucle, <code>poll(0)</code> , <code>sleep</code> , puis <code>flush</code> et bilan	identifiants patients <code>P001</code> ... <code>P0NN</code> , sur <code>NB_PATIENTS</code>

Décisions de configuration, à reporter dans `MESURES.md` :

Paramètre	Ce qu'il règle	Valeur retenue
<code>bootstrap.servers</code>	point d'entrée de découverte du cluster	depuis l'environnement
<code>client.id</code>	nom affiché côté broker	<code>vitals-simulator-s1</code>
<code>acks</code>	niveau d'accusé exigé du broker : <code>0</code> aucun, <code>1</code> le leader, <code>all</code> leader + ISR	_____
<code>linger.ms</code>	attente avant expédition d'un lot : plus de débit, plus de latence	_____
<code>retries</code>	réémissions sur erreur transitoire	<code>5</code>
<code>enable.idempotence</code>	un retry ne crée ni doublon ni inversion d'ordre	<code>True</code>
<code>partitioner</code>	fonction qui décide de la partition à partir de la clé	_____
<code>NB_PATIENTS</code>	nombre de lits simulés	<code>12</code>
<code>INTERVAL_S</code>	intervalle entre deux relevés (un moniteur ICU échantillonne environ 1 Hz)	_____

Sur le `partitioner`, une mise en garde qui vaut d'être lue deux fois : le défaut de la bibliothèque Python **n'est pas** celui du client Java. Or les séances suivantes brancheront un consommateur Java (Flink) sur ce même topic, et les deux doivent placer une clé donnée dans la **même** partition, sinon l'ordre par patient est perdu. Deux valeurs sont possibles, une seule est interopérable avec Java : laquelle ?

Extrait sur le point délicat — la publication elle-même, avec sa clé et son rythme :

```
producteur.produce(
    topic=TOPIC,
    key=_____.encode("utf-8"),          # clé de partitionnement, encodage UTF-8 obliga-
    toire
    value=json.dumps(releve).encode("utf-8"), # valeur : le relevé sérialisé
    on_delivery=accuse_reception,
)
producteur.poll(0)          # traite les accusés en attente SANS bloquer la boucle
# ... et une seule fois, à la sortie de la boucle :
restants = producteur.flush(10) # bloque jusqu'à expédition du buffer, 10 s au plus
```

4.4 Exécuter depuis l'hôte

Première exécution hors conteneur, pour itérer vite :

```
python3 -m venv .venv && source .venv/bin/activate # environnement Python isolé
pip install -r producer/requirements.txt
# Utilisez le port que vous avez publié en partie 3
KAFKA_BOOTSTRAP=localhost:_____ VITALS_TOPIC=mimic-vitals python producer/producer.py
```

Laissez tourner une dizaine de secondes, puis `Ctrl-C`. Dans un second terminal, vérifiez l'arrivée :

```
# --max-messages borne la lecture aux 10 derniers messages produits
docker compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic mimic-vitals --property print.key=true --property print.partition=true \
  --max-messages 10 --timeout-ms 10000
```

4.5 Exécuter via Compose, avec dépendance conditionnelle

Complétez le service `producer` dans `docker-compose.yml` pour qu'il attende un broker **réellement prêt** :

```
depends_on:
  kafka:
    condition: _____ # le healthcheck au vert, pas seulement le conteneur démarré
```

Vérifiez aussi que la variable `KAFKA_BOOTSTRAP` du service vaut bien `kafka:9092` (nom de service, listener interne) et non `localhost`.

```
docker compose up -d --build # relancer toute la pile
docker compose logs -f producer # suivre les logs du producteur
```

Expérience à mener — remplacez temporairement la dépendance conditionnelle par la forme courte `depends_on: [kafka]`, puis :

```
docker compose down                # repartir de zéro : le broker doit réellement démarrer
docker compose up -d
docker compose logs producer | head -20
```

Mesure 6 — Complétez :

Forme de <code>depends_on</code>	Le producteur affiche-t-il des erreurs de connexion au démarrage ?	Nombre de lignes d'erreur avant stabilisation
<code>depends_on: [kafka]</code>		
<code>condition: service_healthy</code>		

Remettez ensuite la forme `condition: service_healthy` : c'est celle qui part au rendu.

Partie 5 — Vérification et mesure

5.1 Débit de production

```
# Somme des offsets de fin AVANT la mesure, toutes partitions
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --bootstrap-server localhost:9092 --topic mimic-vitals --time -1
```

Laissez le producteur tourner exactement 60 secondes, puis relevez de nouveau les offsets de fin.

Mesure 7 — Débit mesuré :

Grandeur	Valeur
Somme des offsets de fin avant (t0)	
Somme des offsets de fin après 60 s (t1)	
Débit calculé $(t1 - t0) / 60$	msg/s
Débit annoncé par le producteur à l'arrêt	msg/s
Valeur de <code>INTERVAL_S</code> et de <code>NB_PATIENTS</code> utilisées	

Question 5.1 — Le débit mesuré correspond-il à `NB_PATIENTS / INTERVAL_S` ? Si l'écart dépasse 5 %, proposez une explication. Répondez dans `MESURES.md`.

Refaites une mesure de 60 secondes avec `INTERVAL_S=0.1` :

Grandeur	Valeur
Débit mesuré avec <code>INTERVAL_S=0.1</code>	msg/s
Écart avec la valeur théorique	

5.2 Coupure du broker pendant la production

Producteur en marche dans un terminal, exécutez dans un autre :

```
docker compose stop kafka # arrêter le broker sans détruire ses données
```

Observez les messages du producteur pendant 30 secondes, puis :

```
docker compose start kafka # redémarrer le broker
```

Mesure 8 — Complétez :

Observation	Valeur relevée
Le processus producteur s'arrête-t-il ?	
Message d'erreur exact affiché (première ligne)	

Observation	Valeur relevée
Délai entre <code>stop</code> et la première erreur	s
Après <code>start</code> , la production reprend-elle sans relancer le script ?	
Des messages produits pendant la coupure ont-ils été perdus ?	

Question 5.2 — En vous appuyant sur les paramètres `retries`, `acks` et `enable.idempotence` de votre configuration, expliquez en quatre phrases ce qui se passe côté producteur entre `stop` et `start`, et ce qui limite la durée pendant laquelle il peut tenir sans broker. Répondez dans `MESURES.md`.

Nettoyage

```
docker compose down -v          # conteneurs, réseau ET volumes nommés déclarés dans le fichier
docker compose ps -a           # aucun conteneur du projet ne doit subsister
docker volume ls | grep vitals  # ne doit rien afficher
docker network ls | grep vitals # ne doit rien afficher
docker system df                # espace récupérable : lisez avant de supprimer quoi que ce soit
deactivate                     # sortir de l'environnement Python
```

Question 6.1 — Quelle est la différence entre `docker compose down` et `docker compose down -v` sur les données du broker ? Que se passe-t-il au prochain `up` dans chaque cas ? Répondez dans `MESURES.md`.

Livable

Archive `S1_<NOM>_<Prenom>.zip` contenant le dossier `s1-vitals-stream/` avec :

1. `docker-compose.yml` — tous les blancs remplis, service `kafka` en KRaft et service `producer` avec `condition: service_healthy`.
2. `producer/Dockerfile` — utilisateur non-root, ordre d'instructions optimisé pour le cache.
3. `producer/.dockerignore` et `producer/requirements.txt`.
4. `producer/producer.py` — complet et fonctionnel.
5. `MESURES.md` — les 8 mesures chiffrées, les 8 réponses aux questions, et les décisions retenues (volume, réseau, port hôte, `acks`, `linger.ms`, `partitioner`, `INTERVAL_S`, `interval` / `retries` du health-check).

Ne mettez pas `.venv/` dans l'archive.

Ce que vous emportez vers la séance 2

Le dossier `s1-vitals-stream/` est votre point de départ de la prochaine séance. Vous y ajouterez un **consommateur**, vous augmenterez le nombre de partitions du topic `mimic-vitals`, vous lancerez plusieurs consommateurs dans un même **consumer group** pour observer la répartition des partitions, et vous écrirez les relevés vers un stockage. Le broker, le topic, le format de message et la clé de partitionnement ne changeront pas : ils ont été fixés aujourd'hui, et ils tiendront jusqu'au projet HealthStream des séances 4 à 6. Conservez l'archive : elle sera demandée en début de séance 2.